

Apêndice — Referências ao Código Implementado

> Documento-base para versão Google Docs, DOCX e PDF.

1) Annotations Customizadas

****Status no branch main:**** não foi identificado o pacote `src/main/java/com/airbnbclone/validator/` nem anotações `@ValidEmail` / `@UniqueEmail`.

****Referência equivalente implementada (validação de email na camada de serviço):****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/HospedeService.java#L54>

Linha no repositório: `HospedeService.java:54-55`

```
if (usuarioRepository.findByEmail(req.email().trim().toLowerCase()).isPresent())
    throw new IllegalArgumentException("Email ja cadastrado.");
```

****Explicação técnica:**** a unicidade de email é validada em runtime por consulta ao repositório antes da persistência do hóspede.

2) Reflection API

****Status no branch main:**** não foi identificado o arquivo

`src/main/java/com/airbnbclone/util/ValidationUtil.java` e não há uso de `java.lang.reflect` no código atual.

****Referência de processamento em runtime existente (sem reflection):****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/ReservaService.java#L64>

Linha no repositório: `ReservaService.java:64-71`

```
Query conflictQuery = new Query(
    Criteria.where("propriedade_id").is(req.propriedadeId())
        .and("status").is("confirmada")
        .and("datas.checkin").lt(checkout)
        .and("datas.checkout").gt(checkin)
);
if (mongoTemplate.exists(conflictQuery, Reserva.class))
    throw new ConflictException("Esta propriedade ja esta reservada para estas datas.");
```

****Explicação técnica:**** a validação dinâmica ocorre por critérios montados em runtime via `MongoTemplate`, sem introspecção reflexiva de campos.

3) Collections Framework

3.1 Entidades (`Usuario`, `Local`, `Propriedade`, `Reserva`)

****Usuario.java****

Link:

<https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/model/Usuario.java#L10>

Linha no repositório: `Usuario.java:10-35`

Uso: entidade persistida e manipulada em coleções de serviço/repositório.

****Local.java (equivalente atual: Propriedade.java)****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/model/Propriedade.java#L21>

Linha no repositório: `Propriedade.java:21`

```
private List<String> comodidades;
```

****Reserva.java****

Link:

<https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/model/Reserva.java#L10>

Linha no repositório: `Reserva.java:10-40`

3.2 Uso de `List`, `Set` e `Map` + operações CRUD

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/ReservaService.java#L116>

Linha no repositório: ReservaService.java:116-124

```
Set<String> propriedadeIds = reservas.stream()
    .map(Reserva::getPropriedadeId).collect(Collectors.toSet());
Map<String, Propriedade> propriedadesMap = propriedadeRepository.findAllById(propriedadeIds)
    .stream().collect(Collectors.toMap(Propriedade::getId, p -> p));
```

****Explicação técnica:**** Set evita duplicidade de IDs, Map otimiza busca por chave no mapeamento de respostas, e List é usada para retorno/iteração de coleções.

4) Streams e Lambdas

****filter() + map() + collect() (camadas de serviço):****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/HospedeService.java#L44>

Linha no repositório: HospedeService.java:44-47

```
return usuarioRepository.findById(id)
    .filter(u -> "hospede".equals(u.getTipo()) || "ambos".equals(u.getTipo()))
    .map(this::toResponse);
```

****map() + collect() em agregação funcional:****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/ReservaService.java#L126>

Linha no repositório: ReservaService.java:126-130

```
return reservas.stream().map(r -> {
    Propriedade p = propriedadesMap.get(r.getPropriedadeId());
    PropriedadeResponse propResp = p != null ? propriedadeService.toResponse(p, anfitrioes) : null;
    return toResponse(r, propResp);
}).collect(Collectors.toList());
```

****Explicação técnica:**** as coleções são processadas de forma declarativa, com transformação e filtragem sem loops imperativos explícitos.

5) Padrões de Design

5.1 Repository Pattern

- UsuarioRepository.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/repository/UsuarioRepository.java#L8>

- ReservaRepository.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/repository/ReservaRepository.java#L1>

- PropriedadeRepository.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/repository/PropriedadeRepository.java#L1>

5.2 Service Layer

- UsuarioService.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/UsuarioService.java#L11>

- ReservaService.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/ReservaService.java#L28>

- PropriedadeService.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/PropriedadeService.java#L22>

5.3 DTO Pattern

- ReservaResponse.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/dto/ReservaResponse.java#L3>

- HospedeRequest.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/dto/HospedeRequest.java#L1>

- PropriedadeResponse.java: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/dto/PropriedadeResponse.java#L1>

****Explicação técnica:**** o backend está segmentado em camadas Controller → Service → Repository, com DTOs para contrato de entrada/saída da API.

6) Tratamento de Exceções

****Handler global:****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/controller/GlobalExceptionHandler.java#L10>

Linha no repositório: `GlobalExceptionHandler.java:10-26`

```
@ExceptionHandler({ConflictException.class})
public ResponseEntity<?> handleConflict(ConflictException e) {
    return ResponseEntity.status(409).body(Map.of("erro", e.getMessage()));
}
```

****Custom exception implementada:****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/service/ConflictExceptionHandler.java#L3>

Linha no repositório: `ConflictExceptionHandler.java:3-6`

****Logging (módulo travelagency):****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/travelagency/service/ViagemService.java#L57>

Linha no repositório: `ViagemService.java:57-65`

```
} catch (ViagemNotFoundException e) {
    logger.error("Erro - Viagem não encontrada: " + e.getMessage());
    throw e;
} finally {
    logger.info("Finalizada busca de viagem com ID: " + id);
}
```

****Observação de nomenclatura:**** no branch atual, as classes `EmailJaExisteException` e `ReservaConflictException` não foram identificadas; o equivalente funcional é `ConflictException`.

7) Serialização JSON

****Status no branch main:**** não há uso explícito de `@JsonProperty` / `@JsonIgnore` nas models atuais.

****Configuração Jackson:****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/resources/application.properties#L5>

Linha no repositório: `application.properties:5`

`spring.jackson.property-naming-strategy=SNAKE_CASE`

****Serialização/desserialização via DTO + @RequestBody:****

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/java/com/airbnbclone/controller/ReservaController.java#L31>

Linha no repositório: `ReservaController.java:31-38`

```
@PostMapping("/reservas")
public ResponseEntity<?> criar(@RequestBody ReservaRequest req) {
    ReservaResponse reserva = reservaService.criar(req);
    return ResponseEntity.status(201).body(Map.of("mensagem", "Reserva confirmada!", "reserva", reserva));
}
```

****Explicação técnica:**** o Spring/Jackson serializa records DTO nas respostas e desserializa JSON de entrada em objetos de requisição.

8) Arquivo de Configuração

****Arquivo:**** `src/main/resources/application.properties`

Link: <https://github.com/CiscG/ProjetoCamadaNegociosf2/blob/main/src/main/resources/application.properties#L1>

Linha no repositório: `application.properties:1-7`

```
spring.application.name=airbnb-clone
spring.data.mongodb.uri=${MONGO_URI:mongodb://host.docker.internal:27017/airbnb_clone}
server.port=5000
spring.web.resources.static-locations=file:FrontEnd/dist/
spring.jackson.property-naming-strategy=SNAKE_CASE
logging.level.root=WARN
logging.level.com.airbnbclone=INFO
```

****Explicação técnica:**** o arquivo centraliza nome da aplicação, URI do MongoDB, porta HTTP, entrega de assets do frontend, estratégia de serialização JSON e nível de logs.